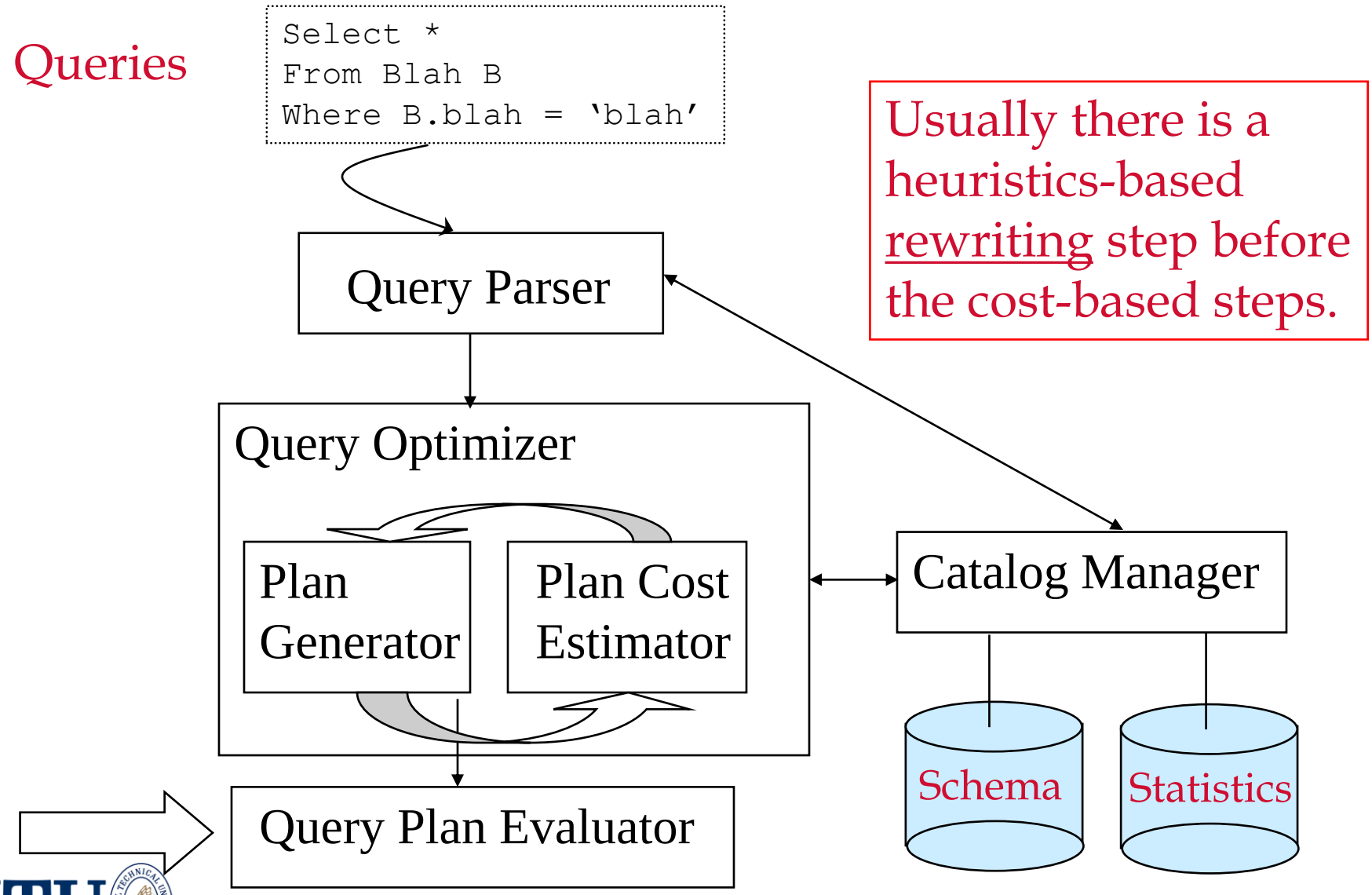
ADVANCED DATABASE SYSTEMS

WEEK 3

Slides credit: S. Shivakumar, Silberschatz, Korth, Sudarshan

QUERY PROCESSING
PART III

Cost-based Query Subsystem



Selection (filter) Operators

Schema for Examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)

Reserves (*sid*: integer, *bid*: integer, *day*: date, *rname*: string)

- Assume pages of 4000 bytes each
- Reserves:
  - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages. (100K reservations)
- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages. (40K sailors)

Simple Selections

```
SELECT *
FROM   Reserves R
WHERE  R.date > '1/1/2015'
```

- Of the form $\sigma_{R.attr\ op\ value}(R)$
- Question: how best to perform?
  - what indexes/access paths are available?
  - what is the expected size of the result (in terms of number of tuples and/or number of pages)?
- Size of result approximated as

  *size of R * reduction factor*

  - "reduction factor" is usually called *selectivity*.
  - estimate of reduction factors is based on statistics – we will discuss shortly.
    - Active research area!

Alternatives for Simple Selections

- With no index, unsorted:
  - Must essentially scan the whole relation
  - cost is M (#pages in R). For "Reserves" = 1000 I/Os.
- With no index, but sorted:
  - cost of binary search + number of pages containing results.
  - For reserves = 10 I/Os + ⌈selectivity*1000⌉
- With an index on selection attribute:
  - Use index to find qualifying data entries,
  - then retrieve corresponding data records.
  - (Hash index useful only for equality selections.)

- Reserves:
  - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages. (100K reservations)
- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages. (40K sailors)

General Selection Conditions

- *(day<8/9/94 AND rname= 'Paul') OR bid=5 OR sid=3*

- Such selection conditions are first converted to *conjunctive normal form* (CNF):
  - *(day<8/9/94 OR bid=5 OR sid=3) AND*
    (rname= 'Paul' OR bid=5 OR sid=3)
- A B-tree index *matches* (a conjunction of) terms that involve only attributes in a *prefix* of the search key.
  - Index on *<a, b, c>* matches *a=5 AND b= 3*, but not *b=3*.
- For Hash index, must have all attributes in search key
- Discussion of the case with Ors → (next slide).

Algorithms for Complex Selections

Disjunction: $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \theta n}(r)$.

- **disjunctive selection by union of identifiers**
  - Applicable if *all* conditions have available indices.
    - Otherwise use linear scan.
  - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
  - Then fetch records from file
- **Negation:** $\sigma_{\neg\theta}(r)$
  - Use linear scan on file
  - If very few records satisfy $\neg\theta$, and an index is applicable to θ
    - Find satisfying records using index and fetch from file

Two Approaches to General AND'ed Selections

- First approach: Find the *most selective access path*, retrieve tuples using it, and apply any remaining terms that don't match the index:
  - *Most selective access path:* An index or file scan that we estimate will require the fewest page I/Os.
  - Terms that match this index reduce the number of tuples *retrieved*; other terms are used to discard some retrieved tuples, but do not affect number of tuples/pages fetched.

Most Selective Index - Example

Consider *day < 8/9/94 AND bid=5 AND sid=3*.

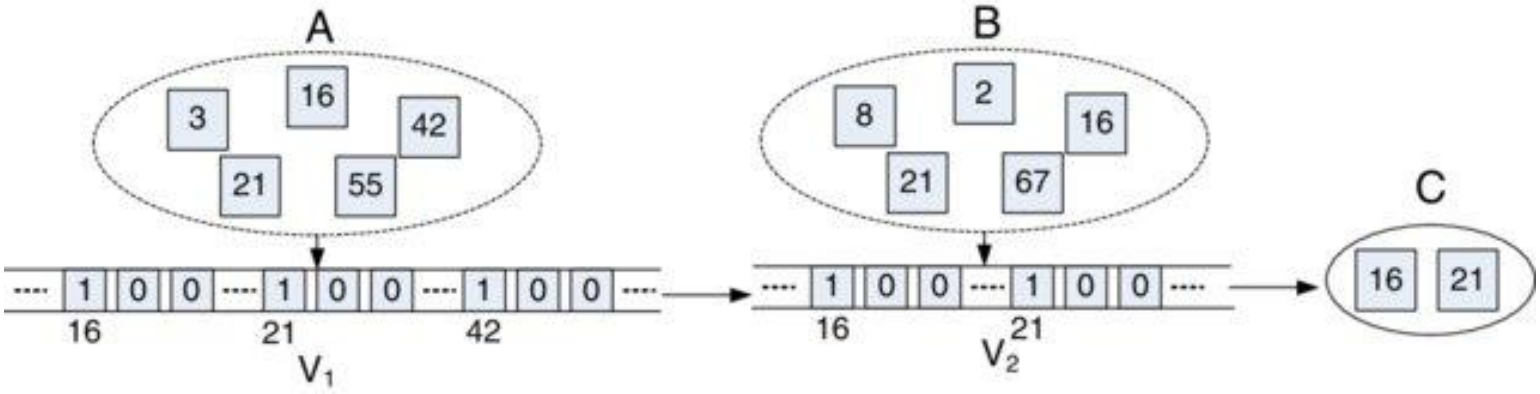
- A B+ tree index on *day* can be used;
  - then, *bid=5* and *sid=3* must be checked for each retrieved tuple.
- Similarly, a hash index on <*bid, sid*> could be used;
  - *Then, day<8/9/94* must be checked.
- *How about a B+tree on <rname,day>?*
- *How about a B+tree on <day, rname>?*
- *How about a Hash index on <day, rname>?*

Intersection of Rids

- Second approach: if we have 2 or more matching indexes :
  - Get sets of rids of data records using each matching index.
  - Then *intersect* these sets of rids.
  - Retrieve the records and apply any remaining terms.
- Consider *day<8/9/94 AND bid=5 AND sid=3*.
  - a B+ tree index on *day* and an index on *sid*
  - retrieve rids of records satisfying *day<8/9/94* using the first,
  - rids of recs satisfying *sid=3* using the second,
  - intersect retrieved rids and check *bid=5*.
  - Note: commercial systems use various tricks to do this:
    - bitmaps, bloom filters, index joins

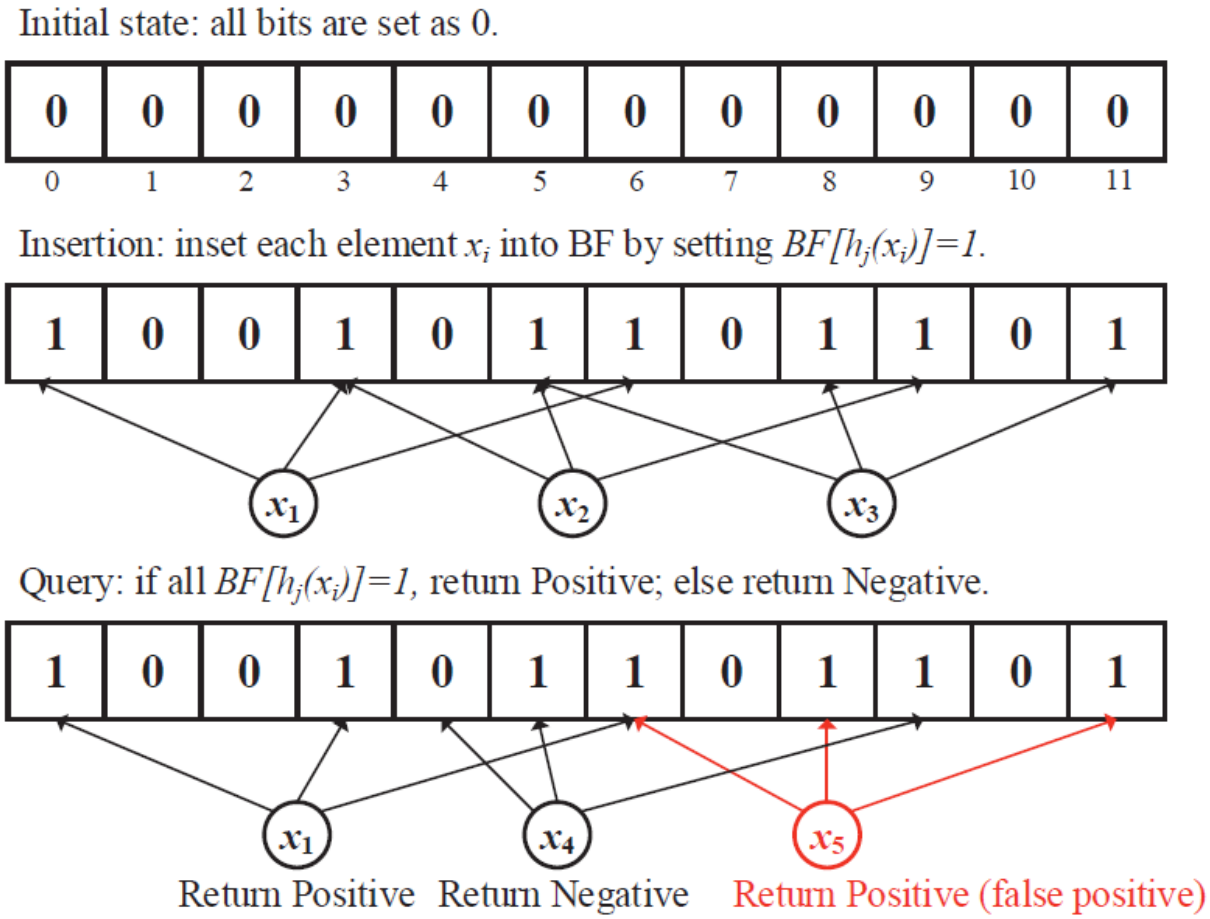
Intersection of Rids

- Bitmaps



Intersection of Rids

- Bloom filters



Index Joins

- Joining indices will become clear soon after covering the join operators.

Join Operators

Join Operators

- Joins are a very common query operation.
- Joins can be very expensive:

  Consider an inner join of R and S each with 1M records.

  Q: How many tuples in the answer?

  (cross product in worst case, 0 in the best(?))

- Many join algorithms have been developed

- Can have very different join costs.

Equality Joins With One Join Column

```
SELECT *
FROM   Reserves R1, Sailors S1
WHERE  R1.sid=S1.sid
```

- In algebra: R ⋈ S. Common! Must be carefully optimized. R x S is large; so, R x S followed by a selection is inefficient.
- Assume:
  - M = 1000 pages in R, p_R =100 tuples per page.
  - N = 500 pages in S, p_S = 80 tuples per page.
  - In our examples, R is Reserves and S is Sailors.
- *Cost metric* : # of I/Os. We will ignore output costs.
- We will consider more complex join conditions later.

Simple Nested Loops Join

```
foreach tuple r in R do
        foreach tuple s in S do
                if r_i == s_j then add <r, s> to result
```

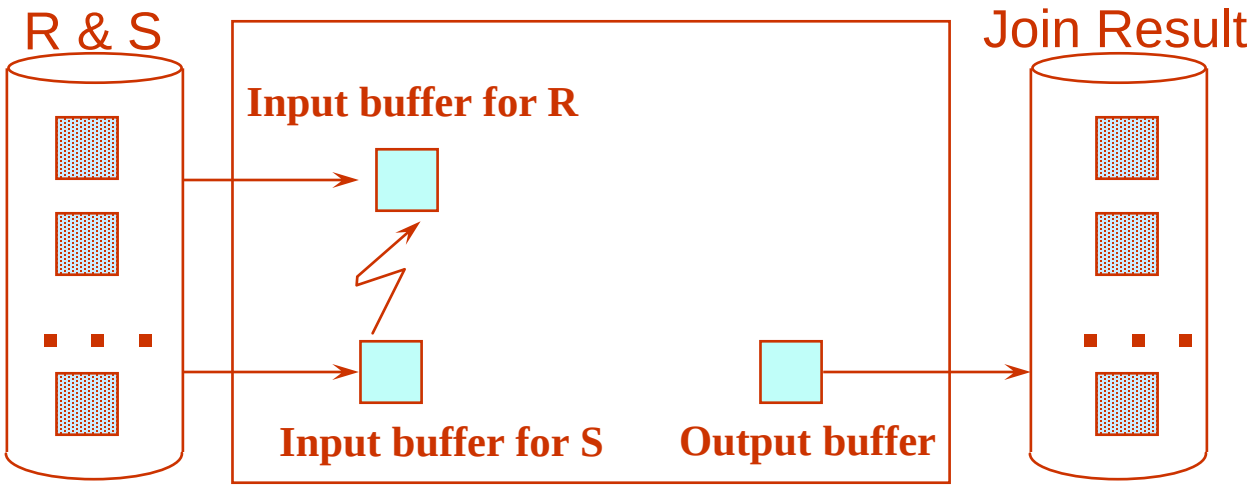
- For each tuple in the *outer* relation R, we scan the entire *inner* relation S.
- How much does this Cost?
- $(p_R * M) * N + M$ = 100,000*500 + 1000 I/Os. (about 50M I/Os!!)
  - At 10ms/IO, Total: ???
- What if smaller relation (S) was outer?
- $(p_s * N) * M + N$ = 40,000*1000 + 500 I/Os. (better…. ☺ 40M I/Os)
- Prohibitively expensive…

Q: What is the cost if one relation can fit entirely in memory?
M+N = 1500 I/Os!!!!!

Page-Oriented Nested Loops Join

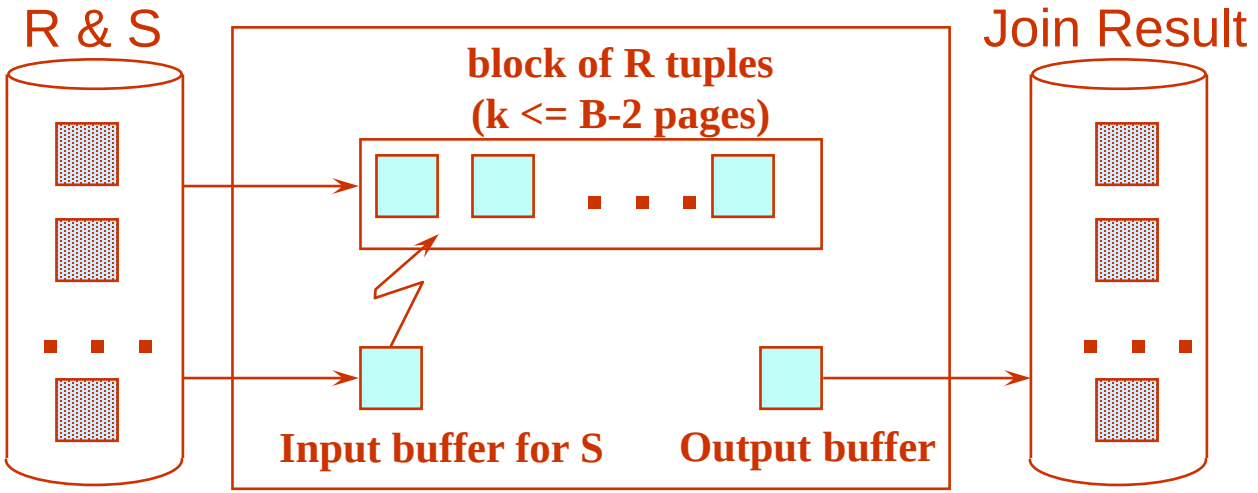
```
foreach page b_R in R do
    foreach page b_S in S do
        foreach tuple r in b_R do
            foreach tuple s in b_S do
                if r_i == s_j then add <r, s> to result
```

- For each *page* of R, get each *page* of S, and write out matching pairs of tuples <r, s>, where r is in R-page and S is in S-page.
- What is the cost of this approach?
- M*N + M = 1000*500 + 1000 = 501000
- If smaller relation (S) is outer, cost = 500*1000 + 500 = 500500



Block Nested Loops Join

- Page-oriented NL doesn't exploit extra buffers.
- Alternative approach: Use one page as an input buffer for scanning the inner S, one page as the output buffer, and use all remaining pages to hold ``block'' of outer R.
- For each matching tuple r in R-block, s in S-page, add <r, s> to result. Then read next R-block, scan S, etc.



Examples of Block Nested Loops

- Cost:
  - Scan of outer + #outer blocks * scan of inner
  - #outer blocks = ceiling(#pages of outer/blocksize)
- With Reserves (R) as outer, and 100 pages/Block (B=102):
  - Cost of scanning R is 1000 I/Os; a total of 10 *blocks*.
  - Per block of R, we scan Sailors (S); 10*500 I/Os.
  - Total cost: 10*500+1000 = 6000 I/Os
  - If space for just 90 pages of R, we would scan S 12 times.
- With 100-page block of Sailors as outer:
  - Cost of scanning S is 500 I/Os; a total of 5 blocks.
  - Per block of S, we scan Reserves; 5*1000 I/Os.
  - Total cost: 5 * 1000 + 500 = 5500 I/Os. (better ☺)
  - We may be able to do even better for different B!

Index Nested Loops Join

foreach tuple r in R do
    foreach tuple s in S where r_i == s_j do
        add <r, s> to result

- If there is an index on the join column of one relation (say S), can make it the inner and exploit the index.
  - Cost: M + ((M*p_R) * cost of finding matching S tuples)
- For each R tuple, cost of probing S index is about 1.2 for hash index, 2-4 for B+ tree.
- Cost of then finding S tuples depends on clustering.
  - Clustered index: 1 I/O per page of matching S tuples.
  - Unclustered: up to 1 I/O per matching S tuple.

Examples of Index Nested Loops

- Hash-index on *sid* of Sailors (as inner):
  - Scan Reserves: 1000 page I/Os, 100*1000 tuples.
  - For each Reserves tuple: 1.2 I/Os to get data entry in index, plus 1 I/O to get (the exactly one) matching Sailors tuple.
  - Total: 1000+100*1000*2.2 = 221K I/Os!
- Hash-index on *sid* of Reserves (as inner):
  - Scan Sailors: 500 page I/Os, 80*500 tuples.
  - For each Sailors tuple: 1.2 I/Os to find index page with data entries, plus cost of retrieving matching Reserves tuples. Assuming uniform distribution, 2.5 reservations per sailor (100,000 / 40,000). Cost of retrieving them is 1 or 2.5 I/Os depending on whether the index is clustered. Assume clustered.
  - Totals: 500 + 80*500*2.2 = 88.5K I/Os!!!
- Not so good here (compared to block nested loops) ☹
- Other scenarios may be better though.

Sort-Merge Join ($R \bowtie_{i=j} S$)

- Sort R and S on the join column, then scan them to do a “merge” (on join col.), and output result tuples.
  - Advance scan of R until current R-tuple >= current S tuple,
  - Then advance scan of S until current S-tuple >= current R tuple;
  - do this until current R tuple = current S tuple.
  - At this point, all R tuples with same value in Ri (*current R group*) and all S tuples with same value in Sj (*current S group*) *match*;
  - output <r, s> for all pairs of such tuples.
  - Then resume scanning R and S.
- R is scanned once;
- Each S group is scanned once per matching R tuple.
  - (Multiple scans of an S group are likely to find needed pages in buffer.)
- Particularly useful if
  - one or both inputs are already sorted on join attribute(s)
  - output is required to be sorted on join attributes(s)

Example of Sort-Merge Join

sid	sname	rating	age
22	dustin	7	45.0
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	bid	day	rname
28	103	12/4/96	guppy
28	103	11/3/96	yuppy
31	101	10/10/96	dustin
31	102	10/12/96	lubber
31	101	10/11/96	lubber
58	103	11/12/96	dustin

- Cost: Sort S + Sort R + (M + N)
  - The cost of merging: usually M + N,
    - worst case is M*N (but very unlikely!)
- With 35, 100 or 300 buffer pages, both Reserves and Sailors can be sorted in 2 passes; total join cost: 7500.

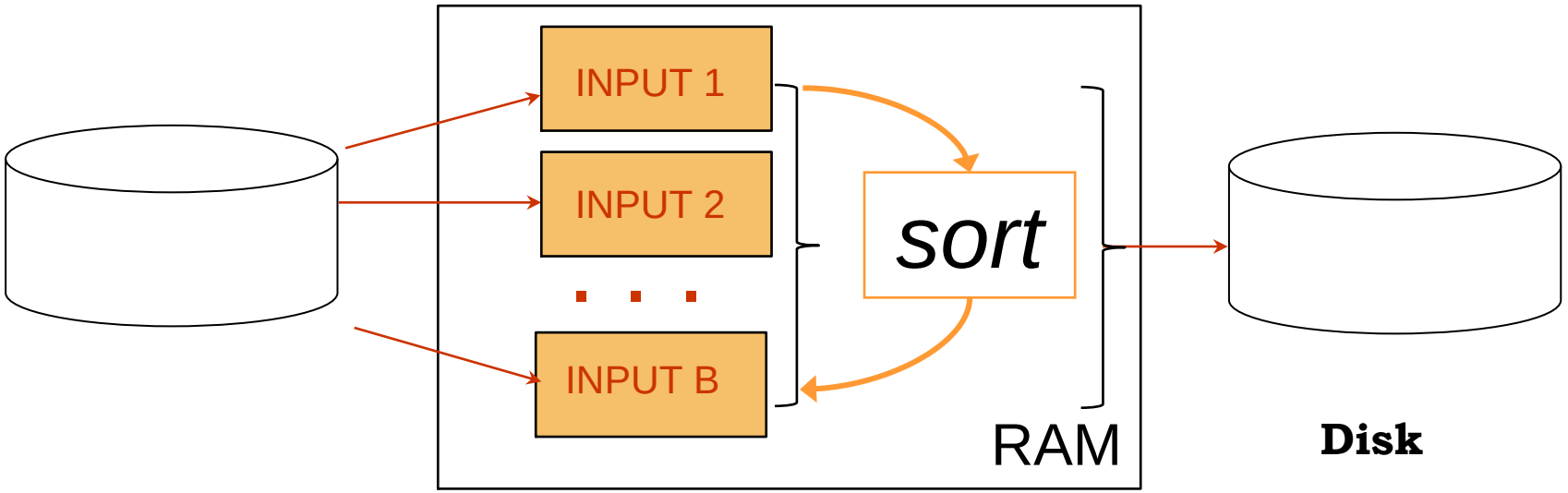
(*BNL cost: 2500 to 16500 I/Os*)

Cost of Sort-Merge

- For B = 35
  - Sort-Merge:
    - sort R: in two passes => 4M = 4000
    - sort S: in two passes => 4N = 2000
    - merge: M + N (hopefully…) => 1500
    - Total: 7500
  - Block Nested Loop Join:
    - celing(N/B-2)*M+N = 16*1000+500 = 16500
  - Sort-Merge Better for B=35!!!!
- For B = 300
  - Sort-Merge: the same: 7500
  - BNLJ:
    - celing(N/B-2)*M+N = 2*1000+500 = 2500
  - Here BNLJ is better!!!!

Resfresher: General External Merge Sort

- More than 3 buffer pages. How can we utilize them?
- To sort a file with N pages using B buffer pages:
  - Pass 0: use B buffer pages. Produce $\lceil N / B \rceil$ sorted runs of B pages each.



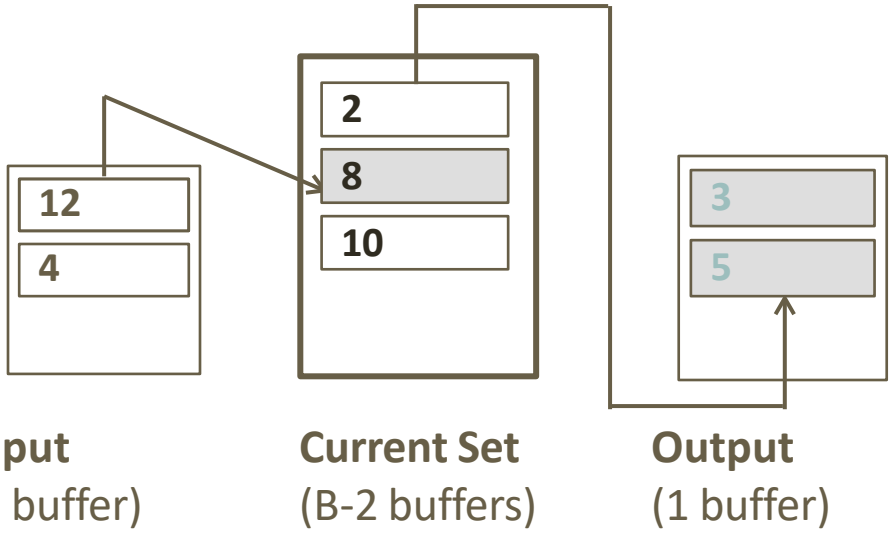
Pass 0 – Create Sorted Runs

Refresher: Internal Sort Algorithm

- Quicksort is a fast way to sort in memory.
- An alternative is “tournament sort” (a.k.a. “heapsort”)
  - `Top:` Read in ***B*** blocks and build a min-heap
  - `Output:` move smallest record to output buffer
  - Read in a new record *r*
  - insert *r* into “heap”
  - if *r is* not the smallest, then `GOTO Output`
  - else remove *r* from “heap”
  - output “heap” in order; `GOTO Top`

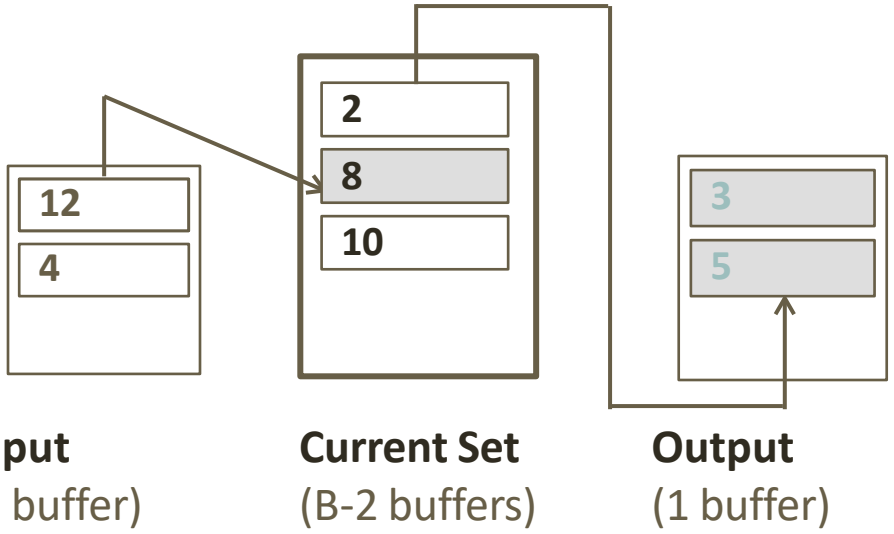
Replacement Sort

- Produces initial sorted runs as long as possible.
- *Replacement Sort*: when used in Pass 0 for sorting, can write out sorted runs of size 2B on average.
  - Affects calculation of the number of passes accordingly.



Replacement Sort

- Organize B available buffers:
  - 1 buffer for *input*
  - B-2 buffers for *current set*
  - 1 buffer for *output*

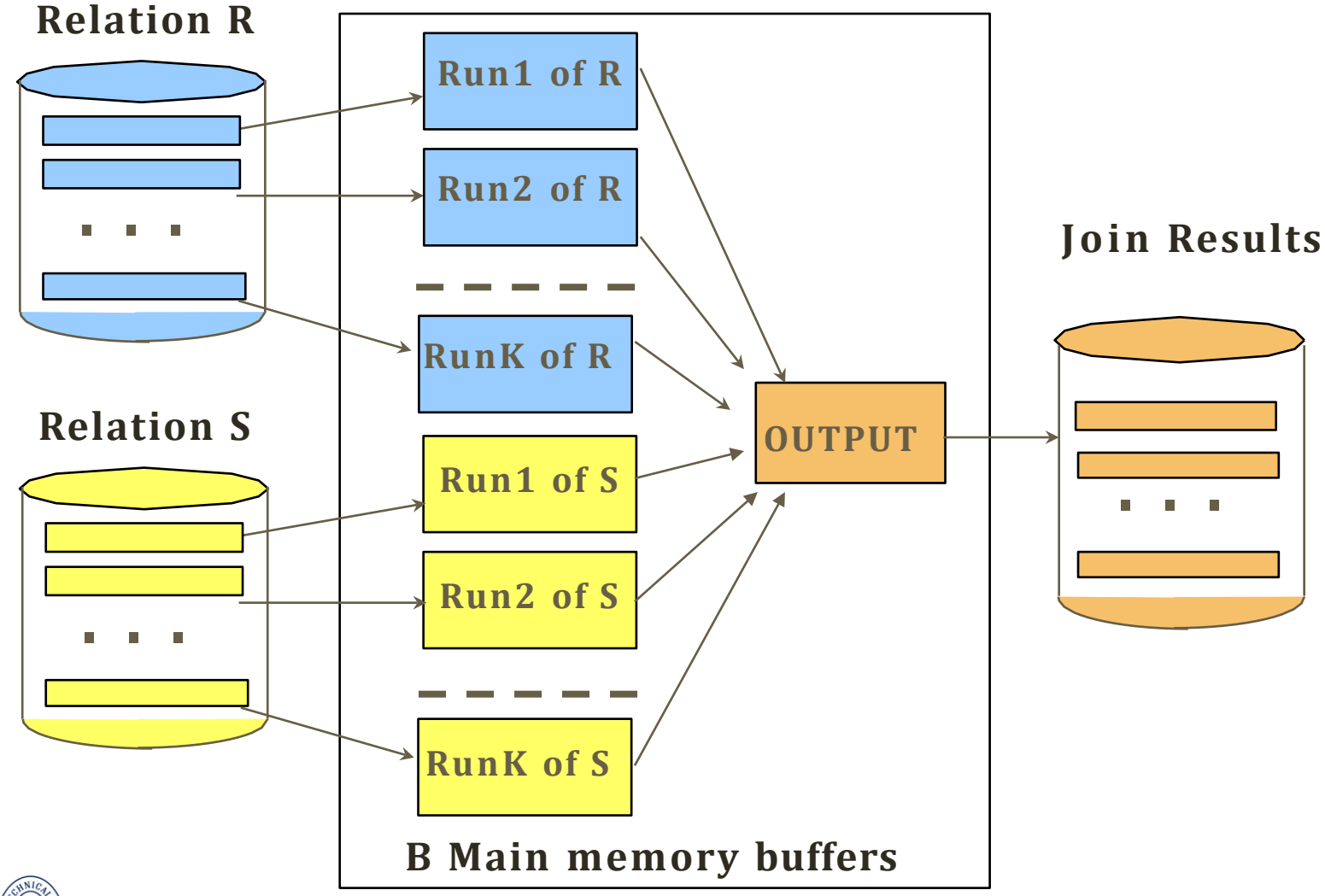


- Pick tuple *r* in the current set with the *smallest value that is ≥ largest value in output*, e.g. 8, to extend the current run.
- Fill the space in current set by adding tuples from input.
- Write output buffer out if full, extending the current run.
- Current run terminates if *every tuple in the current set is smaller than the largest tuple in output*.

Refinement of Sort-Merge Join

- We can combine the merging phases in the *sorting* of R and S with the merging required for the join.
  - Pass 0 as before, but apply to both R then S before merge.
    - If B >=$\sqrt{L}$, where L is the size of the larger relation, using the sorting refinement that produces runs of length 2B in Pass 0, #runs of each relation is < B/2.
  - In "Merge" phase: Allocate 1 page per run of each relation, and 'merge' while checking the join condition
  - Cost: read+write each relation in Pass 0 + read each relation in (only) merging pass (+ writing of result tuples).
  - In example, cost goes down from 7500 to 4500 I/Os for B=300.
- In practice, the I/O cost of sort-merge join, like the cost of external sorting, is *linear*.

Refinement of Sort-Merge Join



Hash-Join

- Partition both relations on the join attributes using hash function h.
- R tuples in partition R_i will only match S tuples in partition S_i.

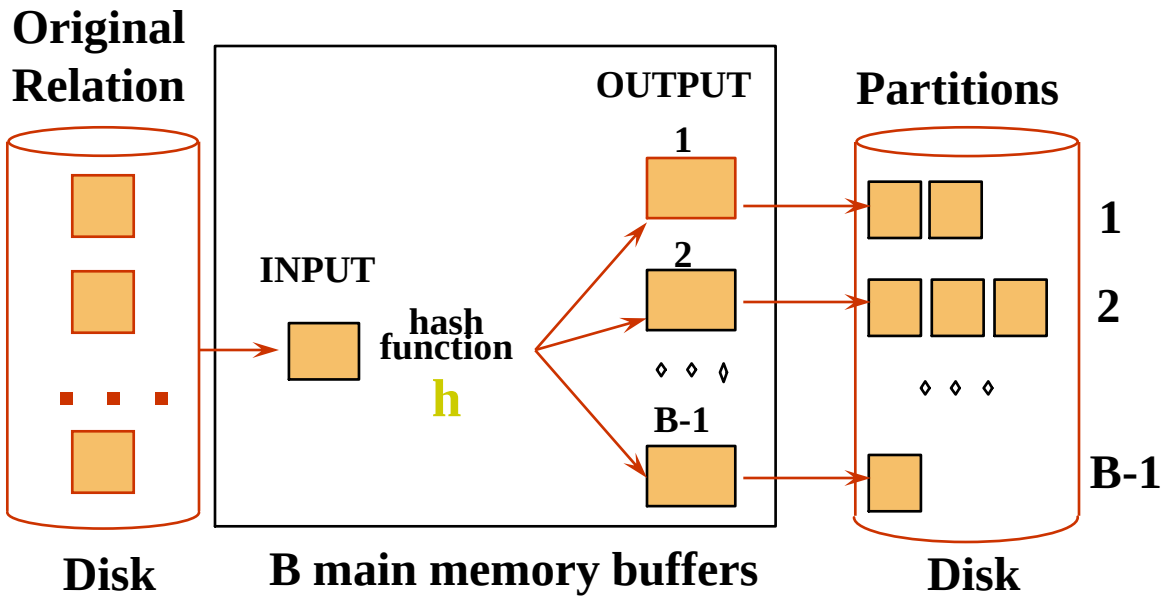
For i= 1 to #partitions {

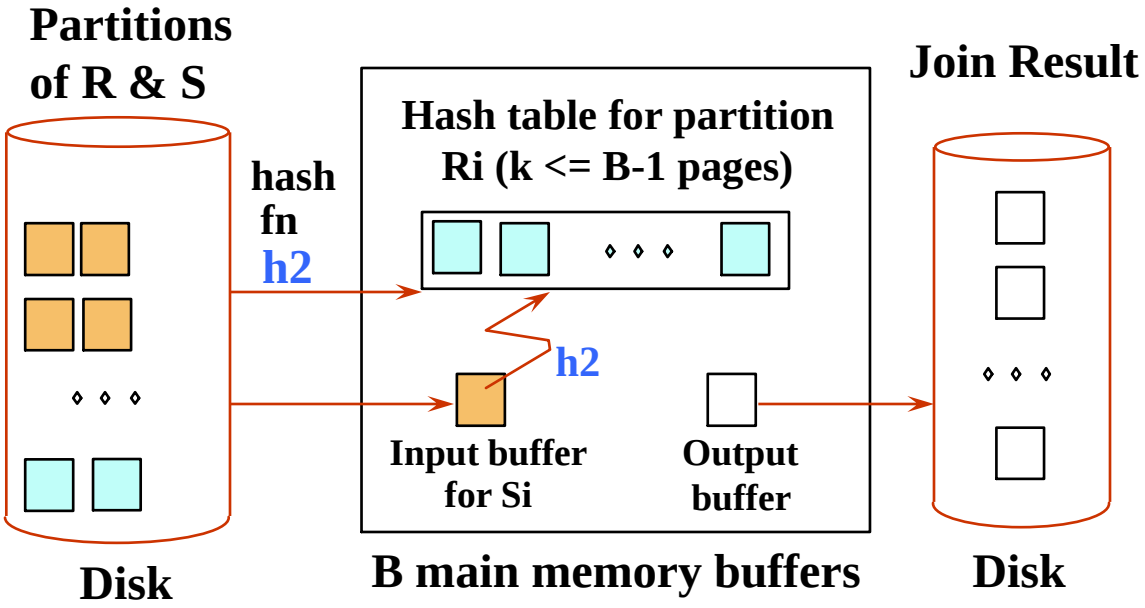
Read in partition R_i

and hash it using h2 (not h).

Scan partition S_i and probe hash table for matches.

}





Observations on Hash-Join

- #partitions k < B, and B-2 > size of smaller partition to be held in memory. Assuming uniformly sized partitions, and maximizing k, we get:

  k= B-1, and M/(B-1) < B-2, i.e., B must be > $\sqrt{M}$

- Since we build an in-memory hash table to speed up the matching of tuples in the second phase, a little more memory is needed.

Handling of Overflows

- Partitioning is said to be **skewed** if some partitions have significantly more tuples than some others
- **Hash-table overflow** occurs in partition s_i if s_i does not fit in memory. Reasons could be
  - Many tuples in s with same value for join attributes
  - Bad hash function
- **Overflow resolution** can be done in build phase
  - Partition s_i is further partitioned using different hash function.
  - Partition r_i must be similarly partitioned.
- **Overflow avoidance** performs partitioning carefully to avoid overflows during build phase
  - e.g. partition build relation into many partitions, then combine them
- Both approaches fail with large numbers of duplicates
  - Fallback option: use block nested loops join on overflowed partitions

Cost of Hash-Join

- In partitioning phase, read+write both relns; 2(M+N). In matching phase, read both relns; M+N I/Os.
- In our running example, this is a total of 4500 I/Os.
- Sort-Merge Join vs. Hash Join:
  - Given a minimum amount of memory, both have a cost of 3(M+N) I/Os. Hash Join superior if relation sizes differ greatly (e.g., if one reln fits in memory). Also, Hash Join shown to be highly parallelizable.
  - Sort-Merge less sensitive to data skew; result is sorted.

Hybrid Hash Join

- Same as partitioned hash join except:
  - Joining phase of one of the partitions is included during the partitioning phase.
- **Partitioning phase**:
  - Allocate buffers for smaller relation- one block for each of the M-1 partitions, remaining blocks to partition 1.
  - Repeat for the larger relation in the pass through S.
- **Joining phase**:
  - M-1 iterations are needed for the partitions R2 , R3 , R4 , ...Rm and S2 , S3 , S4 , ...Sm.
  - R1 and S1 are joined during the partitioning of S1, and results of joining R1 and S1 are already written to the disk by the end of partitioning phase.

General Join Conditions

- Equalities over several attributes
  - (e.g., *R.sid=S.sid* AND *R.rname=S.sname*):
    - For Index NL, build index on *<sid, sname>* (if S is inner); or use existing indexes on *sid* or *sname*.
    - For Sort-Merge and Hash Join, sort/partition on combination of the two join columns.

- Inequality conditions (e.g., *R.rname < S.sname*):
  - For Index NL, need (clustered!) B+ tree index.
    - Range probes on inner; # matches likely to be much higher than for equality joins.
  - Hash Join, Sort Merge Join not applicable!
  - Block NL quite likely to be the best join method here.

Complex Joins

- Join with a conjunctive condition:

$$r \bowtie_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n} s$$

  - Either use nested loops/block nested loops, or
  - Compute the result of one of the simpler joins $r \bowtie_{\theta i} s$
    - final result comprises those tuples in the intermediate result that satisfy the remaining conditions

$$\theta_1 \wedge \ldots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \ldots \wedge \theta_n$$

- Join with a disjunctive condition

$$r \bowtie_{\theta 1 \vee \theta 2 \vee \ldots \vee \theta n} s$$

  - Either use nested loops/block nested loops, or
  - Compute as the union of the records in individual joins $r \bowtie_{\theta i} s$:

$$(r \bowtie_{\theta 1} s) \cup (r \bowtie_{\theta 2} s) \cup \ldots \cup (r \bowtie_{\theta n} s)$$

Outer Joins

- **Outer Join Operators**:
  - **LEFT OUTER JOIN**
  - **RIGHT OUTER JOIN**
  - **FULL OUTER JOIN**.
- The full outer join produces a result which is equivalent to the union of the results of the left and right outer joins.
- Example:

```
SELECT    FNAME, DNAME
FROM      (EMPLOYEE LEFT OUTER JOIN DEPARTMENT
          ON DNO = DNUMBER);
```

- Note: The result of this query is a table of employee names and their associated departments. It is similar to a regular join result, with the exception that if an employee does not have an associated department, the employee's name will still appear in the resulting table, although the department name would be indicated as null.

Outer Joins

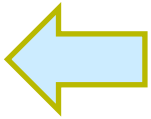
- Executing a combination of relational algebra operators.
- Implement the previous left outer join example
  - {Compute the JOIN of the EMPLOYEE and DEPARTMENT tables}
    - TEMP1 $\leftarrow \pi_{FNAME,DNAME}$(EMPLOYEE $\bowtie_{DNO=DNUMBER}$ DEPARTMENT)
  - {Find the EMPLOYEEs that do not appear in the JOIN}
    - TEMP2 $\leftarrow \pi_{FNAME}$ (EMPLOYEE) - π_{FNAME} (Temp1)
  - {Pad each tuple in TEMP2 with a null DNAME field}
    - TEMP2 ← TEMP2 x 'null'
  - {UNION the temporary tables to produce the LEFT OUTER JOIN}
    - RESULT ← TEMP1 ∪ TEMP2
- The cost of the outer join, as computed above, would include the cost of the associated steps (i.e., join, projections and union).

Outer Joins

- **Modifying Join Algorithms**:
  - Nested Loop or Sort-Merge joins can be modified to implement outer join. E.g.,
    - For left outer join, use the left relation as outer relation and construct result from every tuple in the left relation.
    - If there is a match, the concatenated tuple is saved in the result.
    - However, if an outer tuple does not match, then the tuple is still included in the result but is padded with a null value(s).

Set Operations

- Intersection and cross-product special cases of join.
  - SELECT R.A, R.B
    FROM R, S
    WHERE R.A = S.A and R.B = S.B



```
SELECT R.A, R.B
FROM R
INTERSECT
SELECT S.A, S.B
FROM S
```

- Union (Distinct) and Except similar; we' ll do union.
- Sorting based approach to union:
  - Sort both relations (on combination of all attributes).
  - Scan sorted relations and merge them.
  - *Alternative:* Merge runs from Pass 0 for *both* relations.

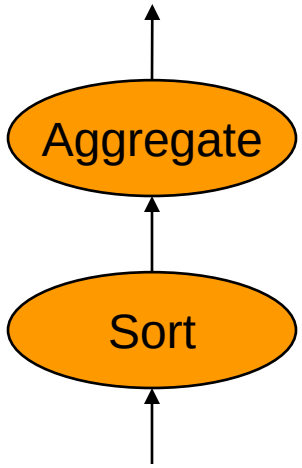
- Hash based approach to union:
  - Partition R and S using hash function *h*.
  - For each S-partition, build in-memory hash table (using *h2*), scan corr. R-partition and add tuples to table while discarding duplicates.

Aggregation Operators

Aggregate Operations (AVG, MIN, etc.)

- Without grouping:
  - In general, requires scanning the relation (FULL SCAN or HEAP SCAN).
  - Given a tree index whose search key includes all attributes in the SELECT and WHERE clauses, can do index-only scan.
- With grouping:
  - Sort on group-by attributes, then scan relation and compute aggregate for each group.
  - Similar approach based on hashing on group-by attributes.
  - Given a tree index whose search key includes all attributes in SELECT, WHERE and GROUP BY clauses, can do index-only scan; if group-by attributes form prefix of search key, can retrieve data entries/tuples in group-by order.

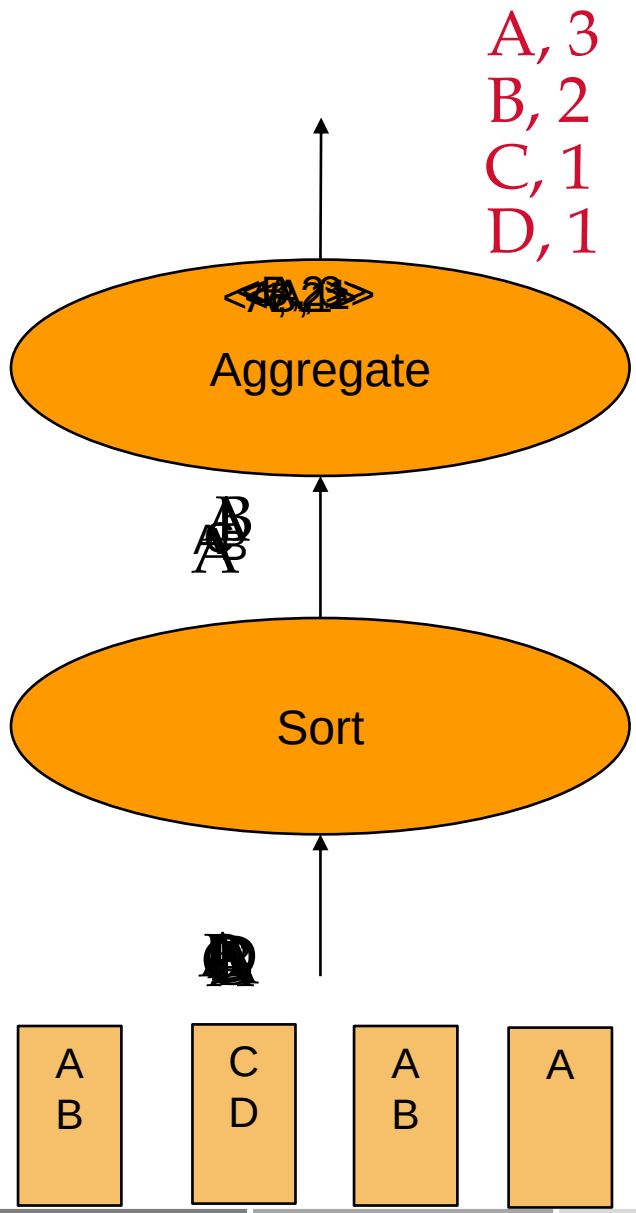
Sort-based GROUP BY Naïve Solution



- The Sort iterator naturally permutes its input so that all tuples are output in sequence
- The Aggregate iterator keeps running info ("transition values" or "transVals") on agg functions in the SELECT list, per group: Example transVals:
  - For COUNT, it keeps `count-so-far`
  - For SUM, it keeps `sum-so-far`
  - For AVERAGE it keeps `sum-so-far` *and* `count-so-far`
- As soon as the Aggregate iterator sees a tuple from a new group:
  1. It produces an output for the old group based on the agg function

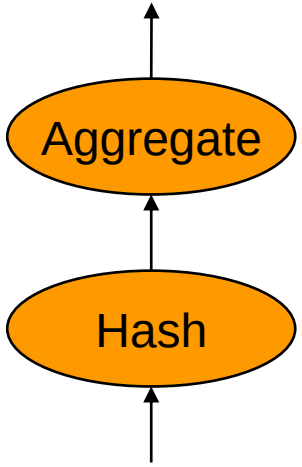
     e.g. for AVERAGE it returns (`sum-so-far/count-so-far`)
  2. It resets its running info.
  3. It updates the running info with the new tuple's info

Sort-based GROUP BY: Naïve Solution



Hash-based GROUP BY
Naïve Solution

- Similar to the Sort-based GROUPBY
  - The Hash iterator → all tuples are output in groups.
- The Aggregate iterator keeps running info ("transition values" or "transVals") on aggregation functions in the SELECT list per group:
- Example transVals:
  - For COUNT, it keeps `count-so-far`
  - For SUM, it keeps `sum-so-far`
  - For AVERAGE it keeps `sum-so-far` and `count-so-far`
- As soon as the Aggregate iterator sees a tuple from a new group:
  1. Produce an output for the old group based on the agg function
     e.g. for AVERAGE it returns (`sum-so-far/count-so-far`)
  2. Reset running info
  3. Update the running info with the new tuple's info

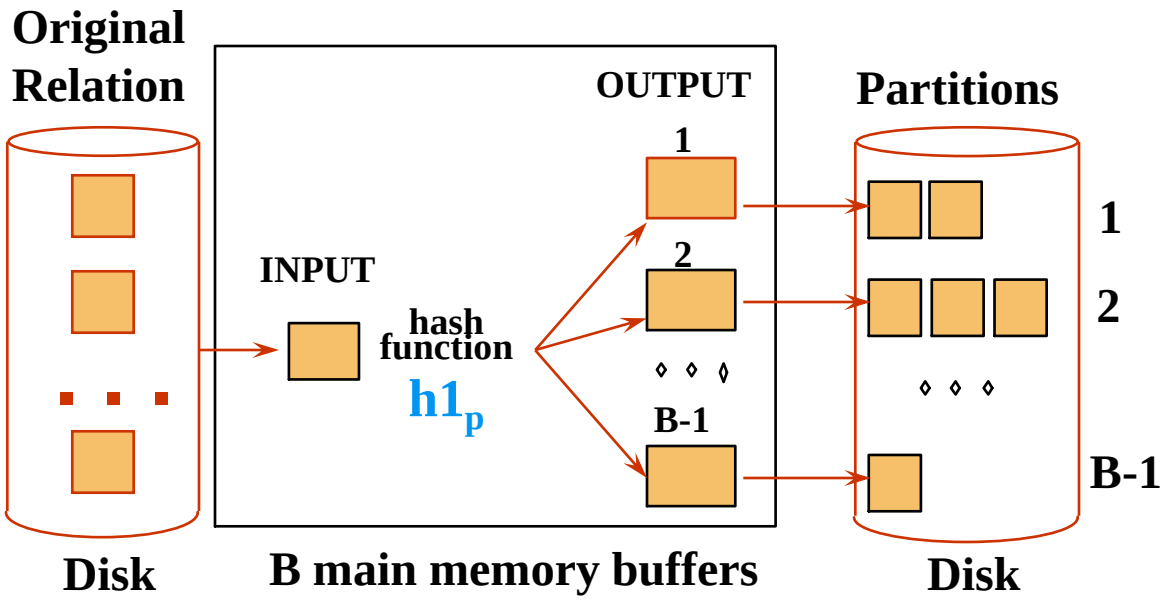


The same as the sort-based group by

External Hashing

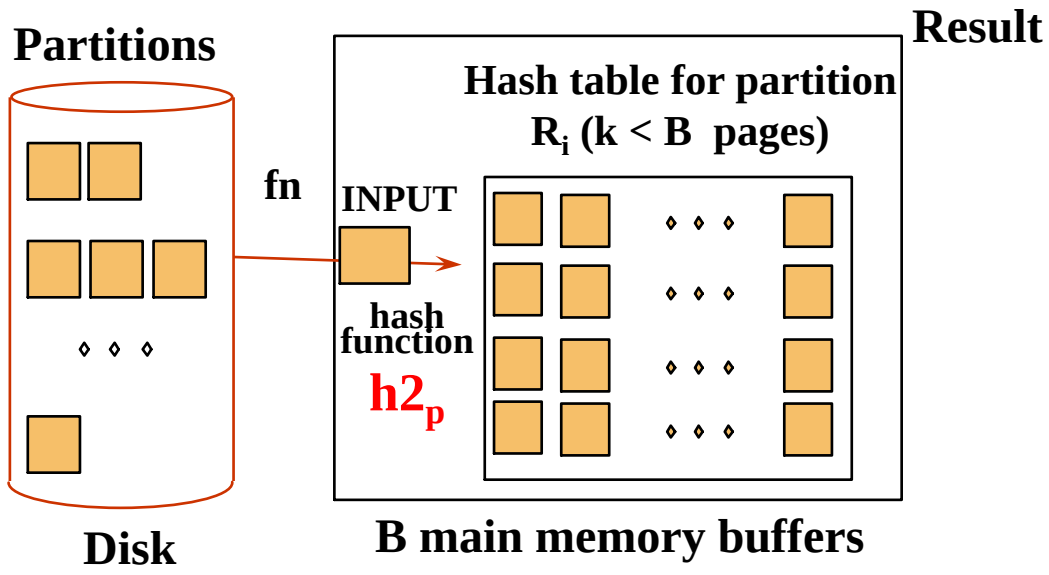
Partition:

- Each group will be in a single disk-based partition file. But those files have many groups inter-mixed.

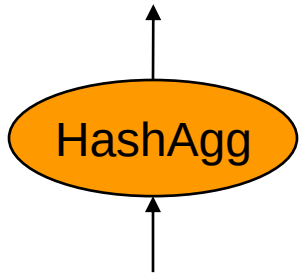


Rehash:

- For Each Partition i:
  - hash i into an in-memory hash table
  - Return results until records exhausted then i++



We Can Do Better!



- Put summarization into the hashing process
  - During the ReHash phase, don't store tuples, store pairs of the form <GroupVals, TransVals>
  - When we want to insert a new tuple into the hash table
    - If we find a matching GroupVals, just update the TransVals appropriately
    - Else insert a new <GroupVals,TransVals> pair
- What's the benefit?
  - Q: How many pairs will we have to maintain in the rehash phase?
  - A: Number of **distinct values** of GroupVals columns
    - Not the number of tuples!!
  - Also probably "narrower" than the tuples

Projection (DupElim)

```
SELECT DISTINCT R.sid, R.bid
  FROM Reserves R
```

- Issue is removing duplicates.
- Basic approach is to use sorting
  1. Scan R, extract only the needed attrs (why do this 1st?)
  2. Sort the resulting set
  3. Remove adjacent duplicates
  - Cost: Reserves with size ratio 0.25 = 250 pages. With 20 buffer pages, can sort in 2 passes, so
    1000 +250 + 2 * 2 * 250 + 250 = 2500 I/Os
- Can improve by modifying external sort algorithm:
  - Modify Pass 0 of external sort to eliminate unwanted fields.
  - Modify merging passes to eliminate duplicates.
  - Cost: for above case: read 1000 pages, write out 250 in runs of 20 pages, merge runs = 1000 + 250 + 250 = 1500.

DupElim & Indexes

- Sort-based approach is the standard; better handling of skew and result is sorted.
- If an index on the relation contains all wanted attributes in its search key, can do *index-only* scan.
  - Apply projection techniques to data entries (much smaller!)
- If an ordered (i.e., tree) index contains all wanted attributes as *prefix* of search key, can do even better:
  - Retrieve data entries in order (index-only scan), discard unwanted fields, compare adjacent tuples to check for duplicates.
- Same tricks apply to GROUP BY/Aggregation

Review

- Implementation of Relational Operations
  - Focus largely on External algorithms (sorting/hashing)
- Joins
  - Blocked nested loops:
    - simple, exploits extra memory
  - Indexed nested loops:
    - best if one relation small and one indexed
  - Sort/Merge Join
    - good with small amount of memory, bad with duplicates
  - Hash Join
    - fast (if enough memory), bad with skewed data
    - Relatively easy to parallelize

Review

- *Queries are composed of a few basic operators*;
  - The implementation of these operators can be carefully tuned (and it is important to do this!).
  - Operators are "plug-and-play" due to the *Iterator* model.
- Many alternative implementation techniques for each operator; no universally superior technique for most.
- Must consider alternatives for each operation in a query and choose best one.
- Choices depend on indexes, memory, stats,…
- This is part of the broader task of Query Optimization, which we will cover next!